

The journey you've taken

Module 1: Environment setup (25 minutes)

You started by preparing your development environment, installing Python, creating a virtual environment, installing ADK, and configuring API keys or Google Cloud project authentication for Agent Platform. You learned the correct setup sequence and verified everything worked by creating and running your first agent with adk create and adk web.

Select the item to learn more.

Key concepts mastered ✓

- Installing ADK with pip install google-adk
- Creating virtual environments for Python projects
- Configuring .env files with GOOGLE_API_KEY or GEMINI_API_KEY / Google Cloud Project authentication (GOOGLE_CLOUD_PROJECT) after running adk create
- Understanding the generated file structure: agent.py, __init__.py, and .env
- Verifying setup with adk web

Module 2: Understanding your first agent (20 minutes)

You learned the four core parameters that define every agent:

- model
- name
- description
- instruction.

You transformed the default agent into a specialized math tutor by customizing its instruction and discovered the difference between description (for other agents) and instruction (for this agent's behavior).

Select the item to learn more.

Key concepts mastered ✓

- Four core parameters: model, name, description, and instruction
- The root_agent variable convention that ADK tools look for
- Writing effective instructions that define agent personality and behavior
- Understanding when to use description versus instruction
- Testing agents with adk web

Module 3: Other ways to run your agent (15 minutes)

You mastered three different ways to run agents beyond the web interface:

- Terminal execution with adk run,
- REST API deployment with adk api_server
- Programmatic execution in Python code.

Each method serves different needs, from quick testing to deployment onto Agent Platform Runtime or custom production integrations.

Select the item to learn more.

Key concepts mastered

- adk run for terminal-based quick testing
- adk api_server for deploying agents as REST APIs
- Programmatic execution using Runner, sessions, and async event loops
- When to use each execution method
- Running from agent directory versus parent directory

Module 4: Agent configuration with YAML (15 minutes)

You explored a second way to define agents using YAML configuration files instead of Python code. You learned that adk create --type=config generates a root_agent.yaml file, making agent creation accessible to non-programmers. You discovered when to use YAML (simple agents and quick prototyping) versus Python code (complex multi-agent systems, custom tools, and enterprise platform workflows).

Select the item to learn more.

Key concepts mastered

- Creating YAML-based agents with adk create --type=config
- Writing agent configurations in YAML format
- Understanding that YAML agent config is Python-only
- Knowing when to use YAML versus Python for agent definition

- Running YAML agents the same way as Python agents

What you can do now



With the skills from this course, you can:

1. **Create** custom agents for any purpose—tutors, assistants, or creative writers
2. **Run** agents multiple ways depending on your needs—web UI, terminal, API, or programmatically
3. **Define** agents in Python or YAML based on your preference and use case
4. **Build** agent-powered applications using the Runner pattern with sessions and prepare them for deployment onto Agent Platform

Key takeaways

As you move forward, keep these truths in mind:

- **Four core parameters:** Every agent needs a `model`, `name`, `description`, and `instruction`.
- `instruction` **is critical:** This parameter defines how your agent behaves and interacts.
- **Multiple execution methods:** Choose `adk web` for development, `adk run` for quick tests, `adk api_server` for APIs, or programmatic execution for custom apps.
- **Python or YAML:** Use YAML for simplicity and accessibility, and Python for complex systems.
- **Environment configuration matters:** Always configure `.env` with `GOOGLE_API_KEY` or set up your Agent Platform credentials / Google Cloud Project ID after `adk create`.



Community and support

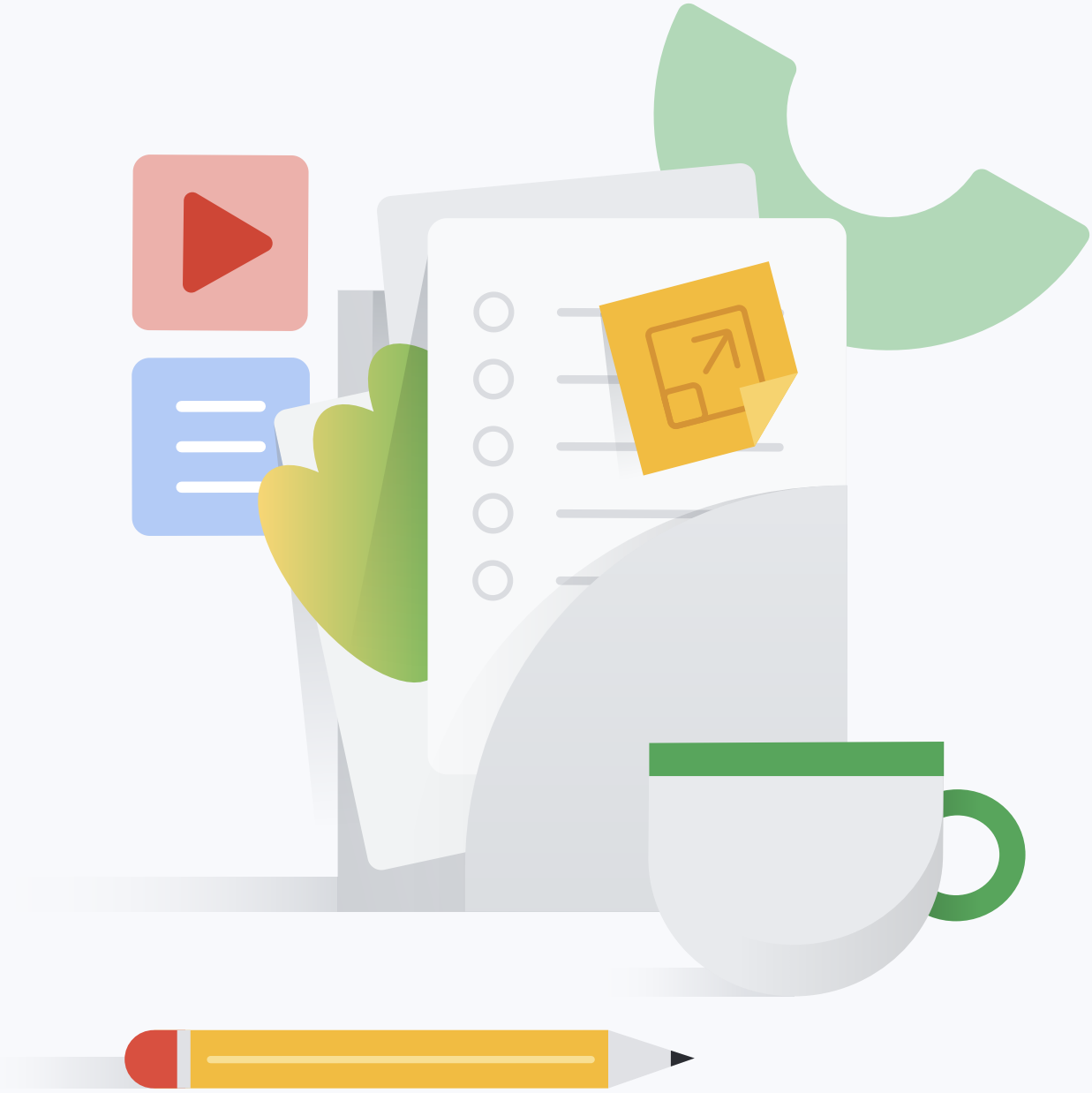
Primary references

- ADK documentation – Official ADK reference
- ADK Python Quickstart – Getting started guide
- Agent Config documentation – YAML configuration reference
- Gemini Enterprise Agent Platform – Agent Platform overview

Google resources

Codelab: Building AI agents with ADK: The foundation ↗

Build an agent with ADK and Agents CLI in Agent Platform ↗



Community resources

- Video: Google Cloud Agent Development Kit tutorial | Build your first ADK agent ↗
- Video: Google AI community training: Getting started with Google Agent Development Kit ↗
- Article: Chapter 1 – Introduction to AI agents and the Agent Development Kit (ADK) ↗
- Course: Hands-on AI: Building agents with the Google Agent Development Toolkit (ADK) ↗

- Build your first AI Agent with ADK – Agent Development Kit by Google ↗
- Your First ADK agent: Building a Google Tasks to-do manager ↗
- Building and deploying a Python ADK agent from the Google Cloud Shell ↗
- Use your ADK AI agent in a UI ↗
- Building and deploying AI agents in minutes with Google's ADK – Part 1 ↗

Questions?

Post in the community forum.

Quick reference card

Essential commands

```
# Environment Setup
python3 -m venv adk-env                # Create virtual environment
source adk-env/bin/activate            # Activate (macOS/Linux)
adk-env\Scripts\activate               # Activate (Windows)
pip install google-adk                 # Install ADK

# Create Agents
adk create my_agent                    # Create Python-based agent
adk create --type=config my_agent      # Create YAML-based agent

# Run Agents
adk web                                # Web interface (from agent dir)
adk web my_agent                       # Web interface (from parent dir)
```

```
adk run                                    # Terminal execution
adk api_server                             # REST API server
```

Python-based agent pattern

```python
from google.adk.agents import Agent

root_agent = Agent(
    model="gemini-flash-latest",
    name="math_tutor_agent",
    description="Helps students learn algebra by guiding them through proble
    instruction="""You are a patient and encouraging algebra tutor.

    Your teaching approach:
    1. When a student asks a question, first understand what they're struggl
    2. Break down the problem into smaller, manageable steps
    3. Guide them to discover the answer rather than giving it directly
    4. Provide positive reinforcement for effort and progress
    5. Use simple language and avoid jargon

    Always maintain a supportive, patient tone."""
)
```

YAML-based agent pattern

```yaml
# root_agent.yaml
name: math_tutor_agent
model: gemini-flash-latest
description: Helps students learn algebra by guiding them through problem-so
instruction: |
  You are a patient and encouraging algebra tutor.

  Your teaching approach:
  1. When a student asks a question, first understand what they're strugglir
  2. Break down the problem into smaller, manageable steps
  3. Guide them to discover the answer rather than giving it directly
```

```
  4. Provide positive reinforcement for effort and progress
  5. Use simple language and avoid jargon

  Always maintain a supportive, patient tone.
```

Programmatic execution pattern

```python
import asyncio
from google.adk.agents import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Create agent
agent = Agent(
    model="gemini-flash-latest",
    name="math_tutor",
    instruction="You are a patient math tutor."
)

# Set up runner
session_service = InMemorySessionService()
runner = Runner(
    agent=agent,
    app_name="my_app",
    session_service=session_service
)

# Run agent
async def run_agent():
    session = await session_service.create_session(
        app_name="my_app",
        user_id="user_1",
        session_id="session_1"
    )

    user_message = Content(
```

```
        role="user",
        parts=[Part.from_text(text="How do I solve 2x + 5 = 13?")]
    )

    async for event in runner.run_async(
        user_id="user_1",
        session_id="session_1",
        new_message=user_message
    ):
        if event.is_final_response() and event.content and event.content.par
            for part in event.content.parts:
                if part.text:
                    print(part.text)

# Execute
asyncio.run(run_agent())
```